

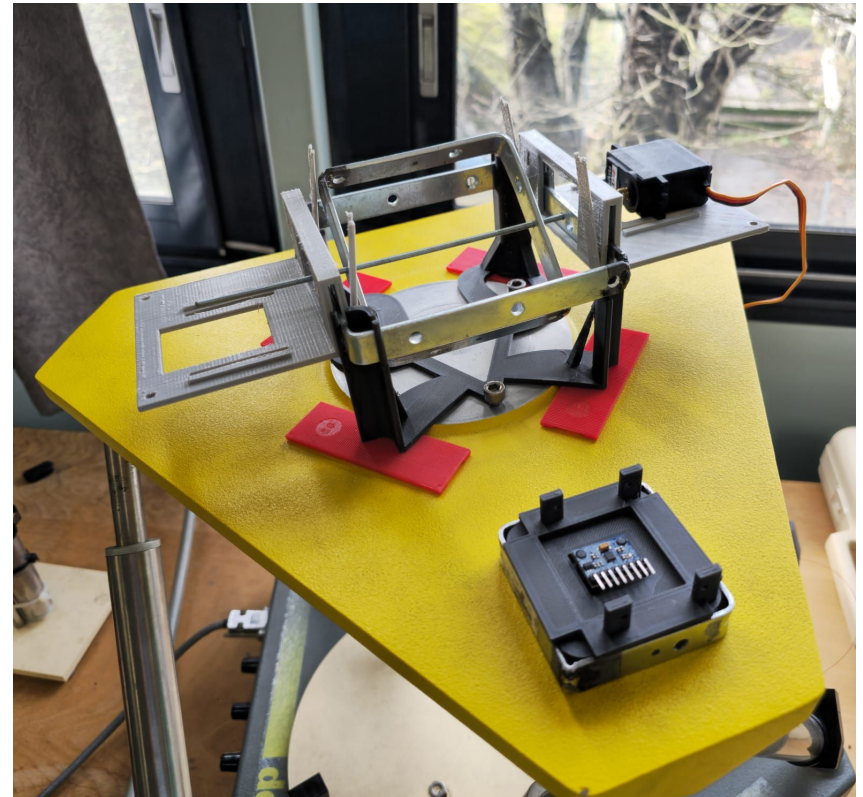
Stabilisation d'une Plateforme Multi-axes

Thème : Cycles et Boucles

PLAN DE LA PRESENTATION

- 1 Contexte et problématique
- 2 Conception (théorique et pratique)
- 3 Architecture électronique & capteurs
- 4 Commande PID & Résultats expérimentaux

TEKOMBONG Steve
Numéro de Candidat : 38989
Session 2025-2026



Contexte : Stabilisation active

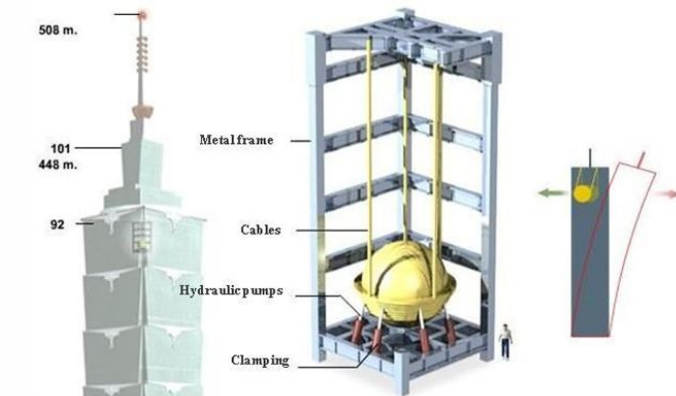


Fig. 1. Damper in Taipei 101 building.

<https://static.longnow.org/2024/03/taipei101-diagram.jpg>
Source: longnow.org

Inspirations industrielles



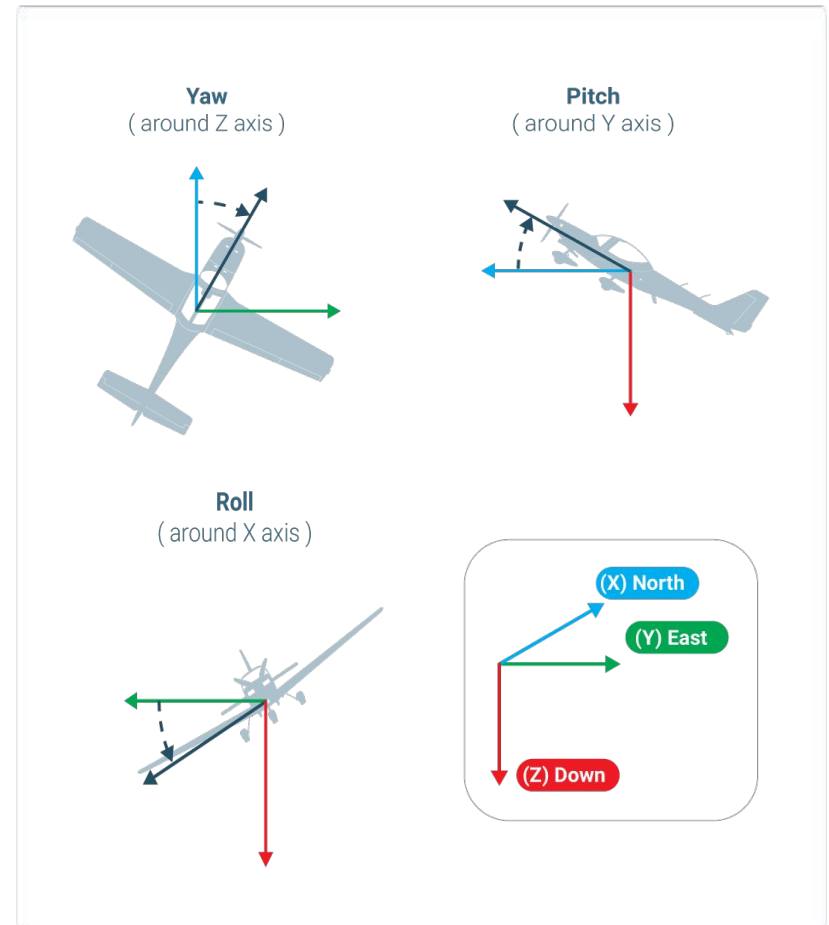
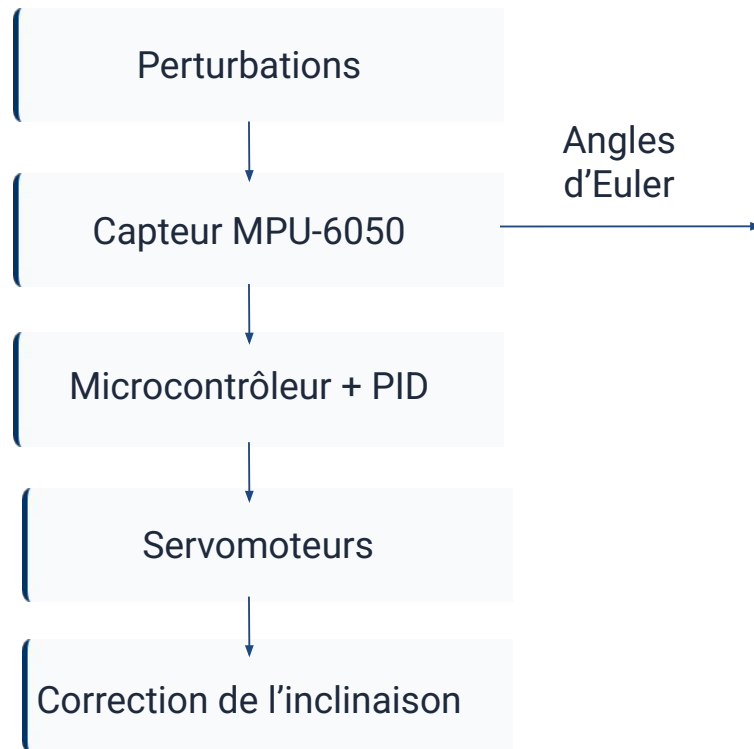
Génie Civil : Amortisseur harmonique (TMD) du Taipei 101 face aux séismes: stabilisation passive



Marine : Plateformes pétrolières compensant l'effet de la houle: stabilisation active

- Traiter et fusionner les données issues d'un capteur inertiel MPU-6050
 - Estimer avec précision l'orientation d'une plateforme multi-axes
 - Piloter un servomoteur via un microcontrôleur
- Assurer un asservissement angulaire stable malgré les perturbations

Principe de l'asservissement



Angles d'Euler : Paramétrage de la rotation du solide (Tangage / Roulis)

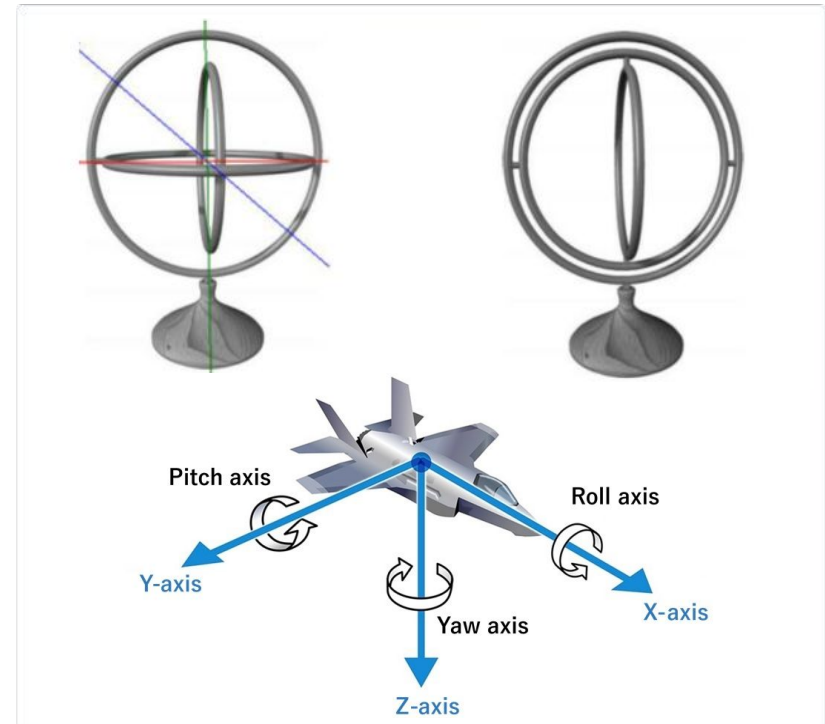
<https://support.pix4d.com/hubfs/Knowledge%20Base%20Import/360047787932.png>

Source: support.pix4d.com

Singularité : Le blocage de cardan (Gimbal Lock)

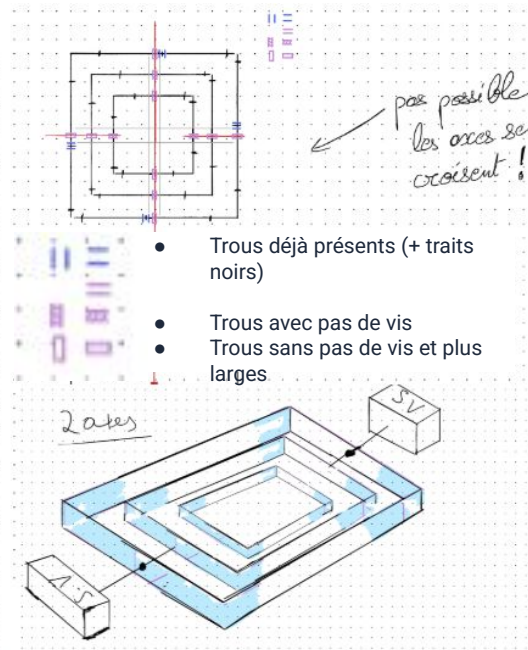
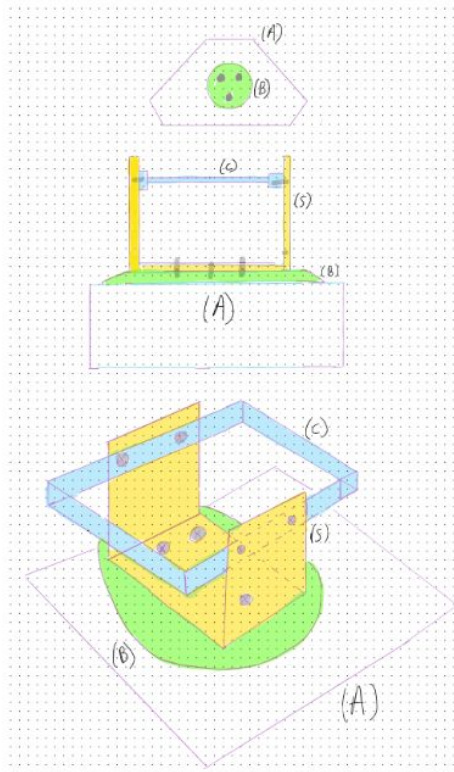
Définition : Perte d'un degré de liberté lorsque deux des trois axes de rotation deviennent colinéaires $\Rightarrow$ impossible de compenser les perturbations

Traitement : Limitation des consignes angulaires en amont de la boucle d'asservissement et utilisation des angles (Yaw, Pitch, Roll)



<https://www.slideserve.com/liuz/animating-human-model-in-opengl-using-data-from-motion-capture-system>
Source: [Slideserve.com](https://www.slideserve.com)

Conception théorique



Schématisation

Uniaxial $\neq$ Biaxial : Attention aux croisements

Prise de mesure : Utilisation d'un pied à coulisse afin d'augmenter la précision

Identification des difficultés : Position des servomoteurs et de la plateforme à stabiliser

Légende

(A) Plateforme soumise aux perturbations

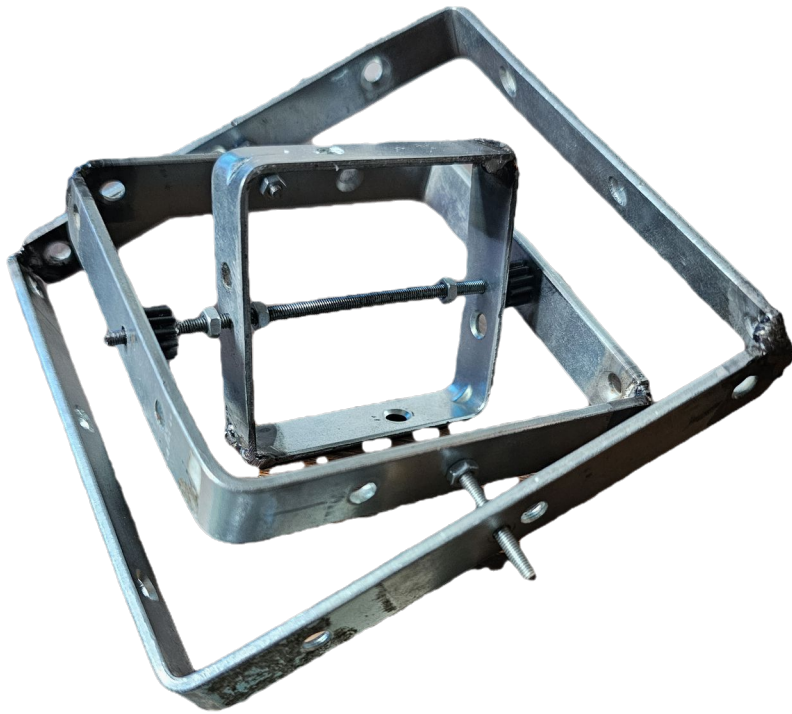
(B) Pièce de fixation du système (3 vis)

(S) U – support du cardan fixe

(C) Cardan fixe

(SV) Servomoteurs

Conception mécanique



Fabrication Personnelle

- 📦 **Soudure** : Soudure d'équerre légère mais robuste
- 🖨️ **Impression 3D** : Supports et engrenages
- ⚙️ **Cardans** : Liaison pivot à deux axes pour une liberté de mouvement totale.

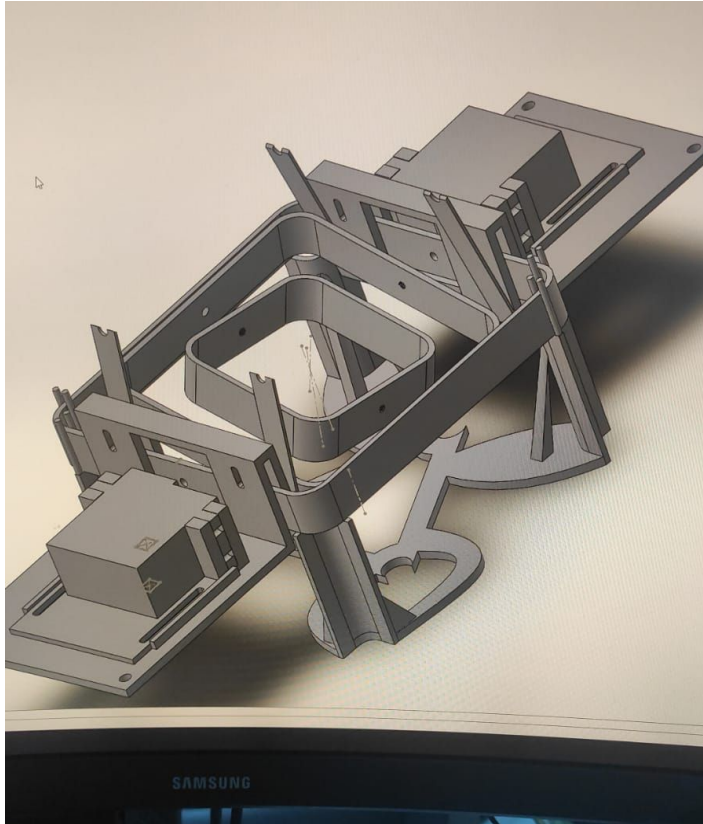
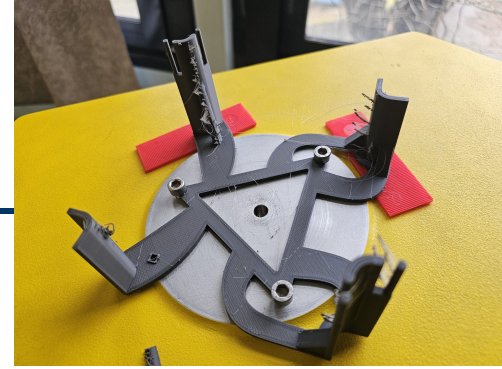
Description physique

- Acier
- Masse de l'ensemble : 364 g
- Tige filetée et écrous de 3 mm de diamètre
- Engrenage en ABS (1)

Tableau des tailles

- Côté du 1er cardan : 12 cm
- Côté du 2eme cardan : 9 cm
- Côté du 3eme cardan : 6 cm

Conception mécanique



Création du support

- 📦 **Modélisation 3D** : Conception sous CAO pour optimiser les jeux et l'intégration des servomoteurs.
- 🖨️ **Impression 3D** : Choix de l'ABS pour la rigidité et la légèreté.

Description physique

- Support principal/servomoteur en ABS gris clair/foncé
- Masse de l'ensemble : 160 g

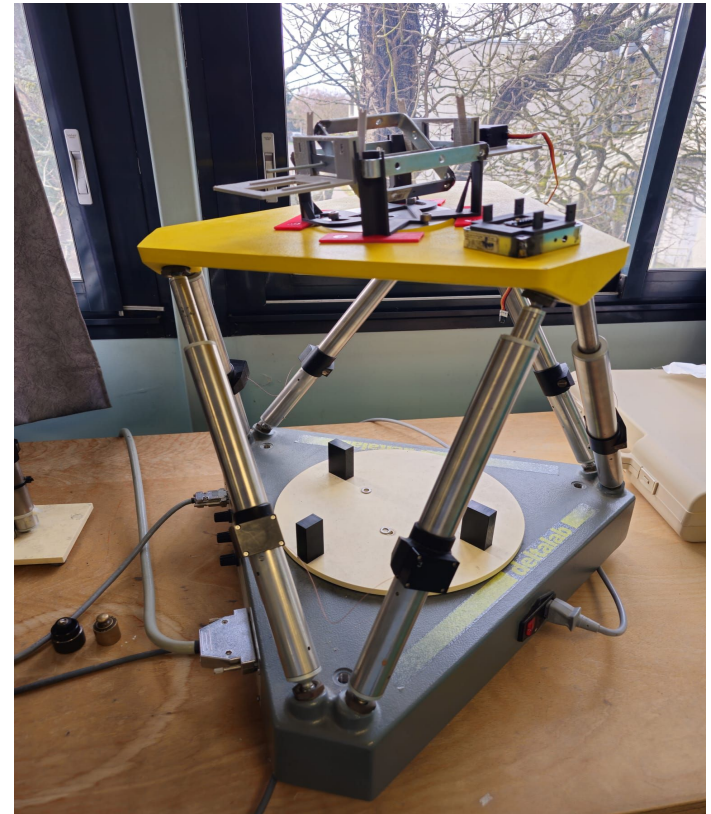
Tableau des tailles

- Côté du support : 13 cm
- Hauteur du support : 6 cm
- Longueur du support pour (SV) : 7cm

Simulation des perturbations

Plateforme de Stewart pilotée

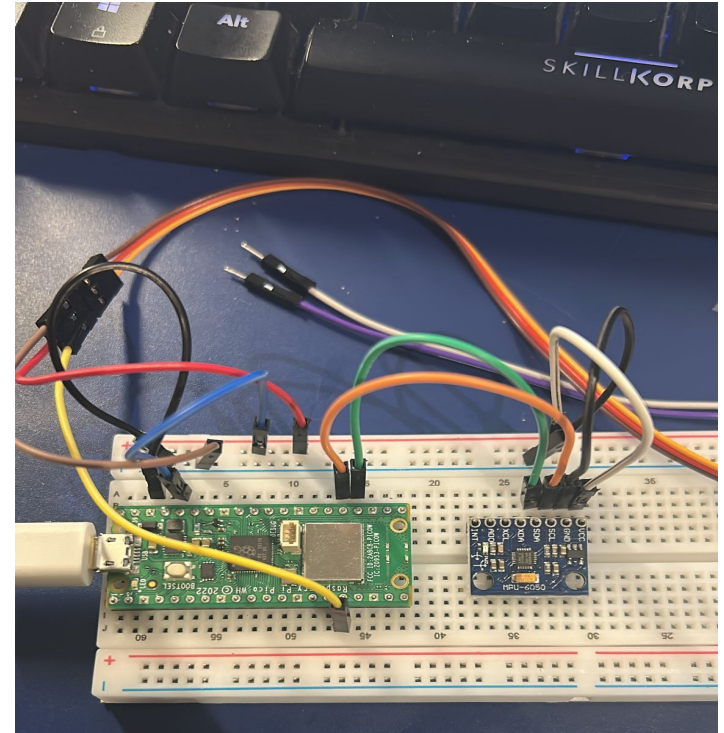
- ⇒ **Simulateur d'environnement** : Trajectoires dynamiques reproductibles (houle, séismes, inclinations)
- ⚙️ **Actionnement Multi-axes** : 6 vérins électriques → 6 degrés de liberté
- 🔊 **Asservissement** : asservissement en position
- 💻 **Pointage logiciel** : Analyse logicielle : consigne vs stabilisation réelle
- ⚙️ **Test de Robustesse** : Validation de la robustesse du PID sous perturbations réelles



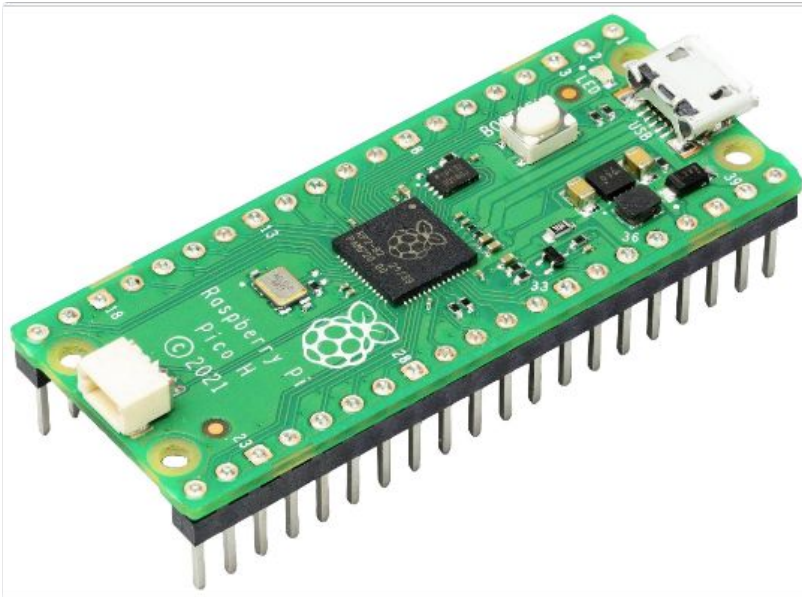
Acquisition : Capteur MPU-6050

Centrale à inertie (IMU)

- 🔧 **Accéléromètre** : Mesure de la gravité (statique).
- 🌀 **Gyroscope** : Mesure des vitesses angulaires (dynamique).
- 🔗 **Protocole I2C** : Transfert de données vers le contrôleur (Raspberry Pi Pico WH)



Traitement : Raspberry Pi Pico WH



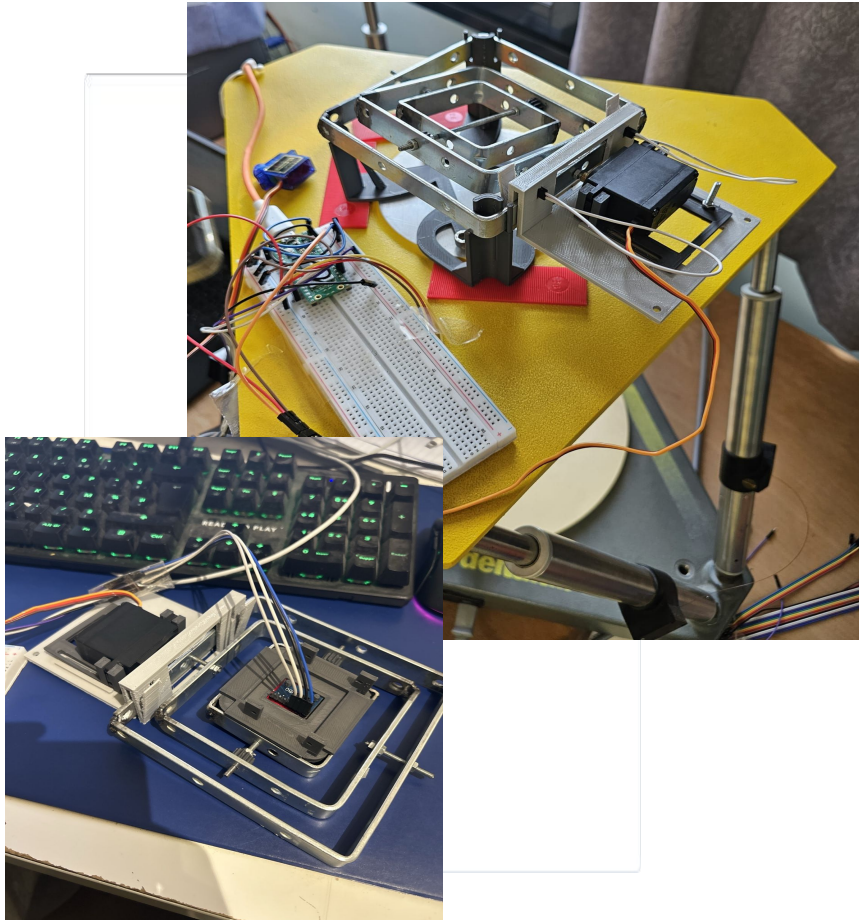
Cerveau du système

- ⚡ **Vitesse de calcul** : Grande rapidité nécessaire à la stabilisation
- 📄 **Programmation** : Implémentation du filtrage et de l'algorithme PID
- 📶 **Pico WH** : Récupération des données en temps réel via le module PySerial

<https://asset.conrad.com/media10/isa/160267/c1/-/en/002615362PI00/image.jpg>

Source: www.conrad.com

Actionneur : Servomoteur



Actionneur

Couple suffisant : Faire tourner les cardans de manière précise

Programmation : Choix de la bonne bande de fréquence, afin de couvrir complètement la plage d'angle $[0; \pi]$

Asservissement : Récupération de la position angulaire courante indispensable à l'asservissement

Programmation : MicroPython

$$\psi = \text{atan2}(2q_2q_3 - 2q_1q_4, 2q_1^2 + 2q_2^2 - 1)$$

$$\theta = -\sin^{-1}(2q_2q_4 + 2q_1q_3)$$

$$\phi = \text{atan2}(2q_3q_4 - 2q_1q_2, 2q_1^2 + 2q_4^2 - 1)$$

Calcul des angles à partir des quaternions

Conversion : quaternions $\Rightarrow$ angles d'Euler

(yaw, pitch, roll) = (psi, phi, theta) via

formules trigonométriques

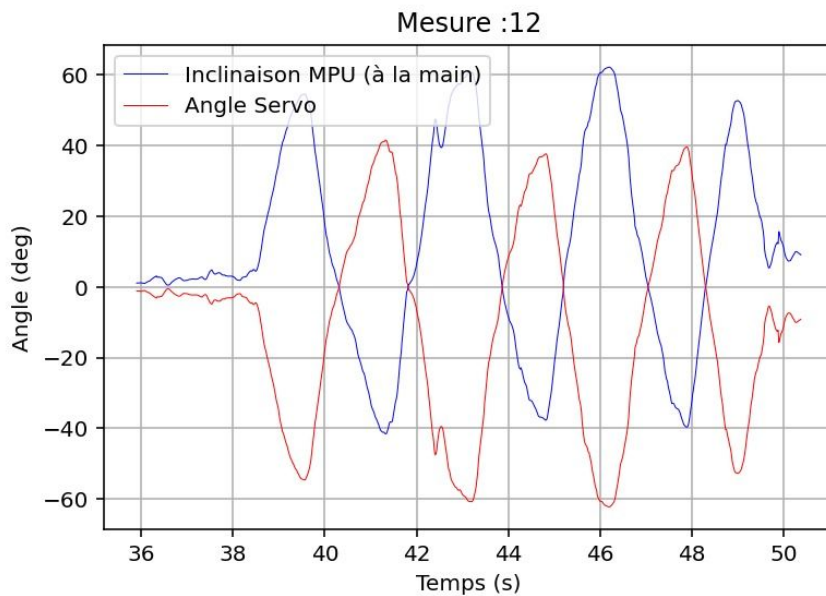
Limite : singularité géométrique (gimbal lock) lorsque le pitch = $\pm 90^\circ$

https://web.enib.fr/~kerhoas/iot/reseau-de-capteurs/carte-imu-mpu9250/documents/INVENSENSE/madgwick_internal_report.pdf

Source: [An efficient orientation filter for inertial and inertial/magnetic sensor arrays](#)

```
def dmpGetEuler(self, quat):
    qw, qx, qy, qz = quat
    psi = atan2(2*qx*qy - 2*qw*qz, 2*qw**2 + 2*qx**2 - 1)
    phi = atan2(2*qy*qz - 2*qw*qx, 2*qw**2 + 2*qz**2 - 1)
    theta = -asin(2*qx*qz + 2*qw*qy)
    return theta, phi, psi
```

Manipulations



Premières manipulations

- 📈 Mise en mouvement du MPU à la main
- 🔧 Tracé du comportement du système sous Python.

Asservissement : Boucle PID

$$u(t) = K_p \varepsilon(t) + K_i \int \varepsilon(t) dt + K_d \frac{d\varepsilon(t)}{dt}$$

Boucle de rétroaction : Correction continue de l'erreur angulaire

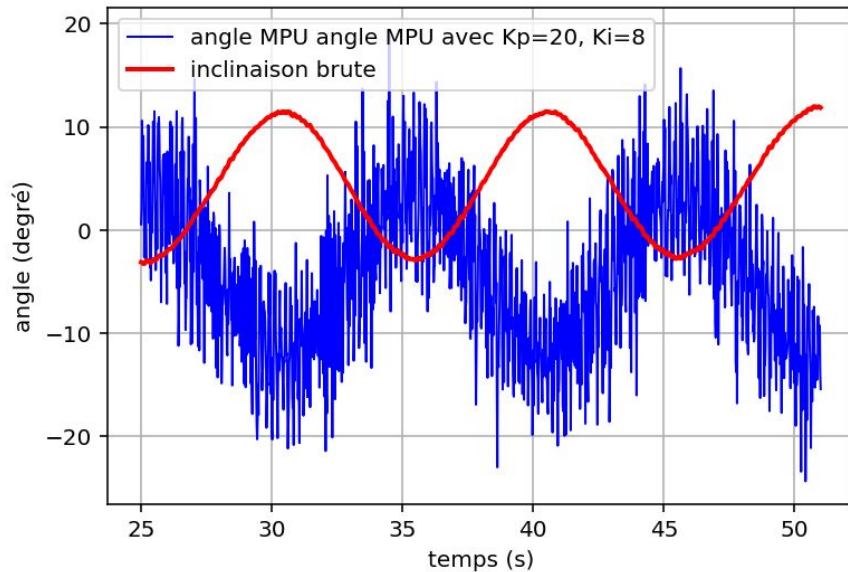
✍ Analyse de l'influence des coefficients

Proportionnel : Réactivité face à l'écart.

Intégral : Annulation de l'erreur statique.

Dérivé : Amortissement des oscillations.

Manipulations



Utilisation de Stewart

- ✍ Mise en mouvement de la plateforme
- 🔧 Présence de bruit, calculs trop fréquents
`sleep_ms(10)`
- ✔ Critère de stabilité : Angle MPU = - Inclinaison brute

Solutions

Fusion de données

Le gyroscope dérive, l'accéléromètre est bruité.

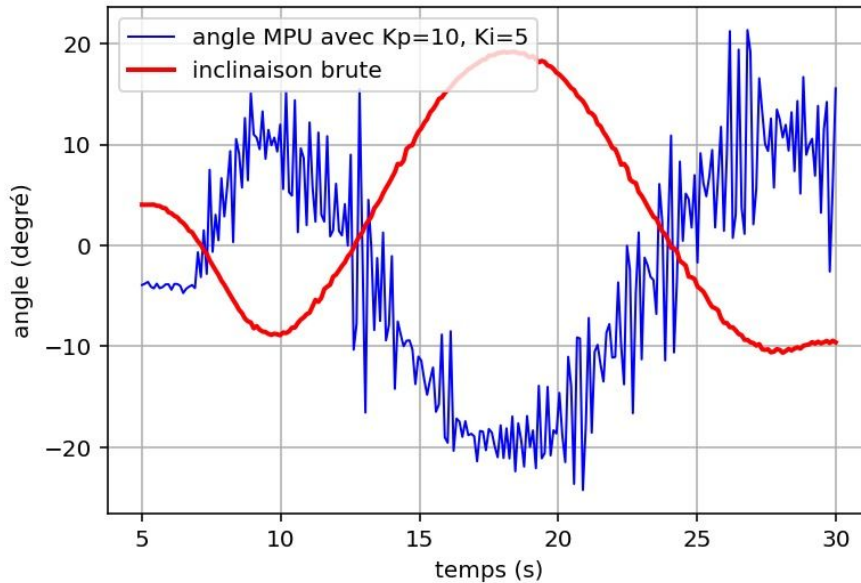
```
pitch_gyro, pitch_acc = mpu.pitch_roll()  
pitch_filtre = (1-alpha)*pitch_gyro + alpha*pitch_acc
```

Cycles : Temps d'échantillonnage constant pour
assurer la stabilité temporelle.



```
sleep_ms(100)
```

Manipulations



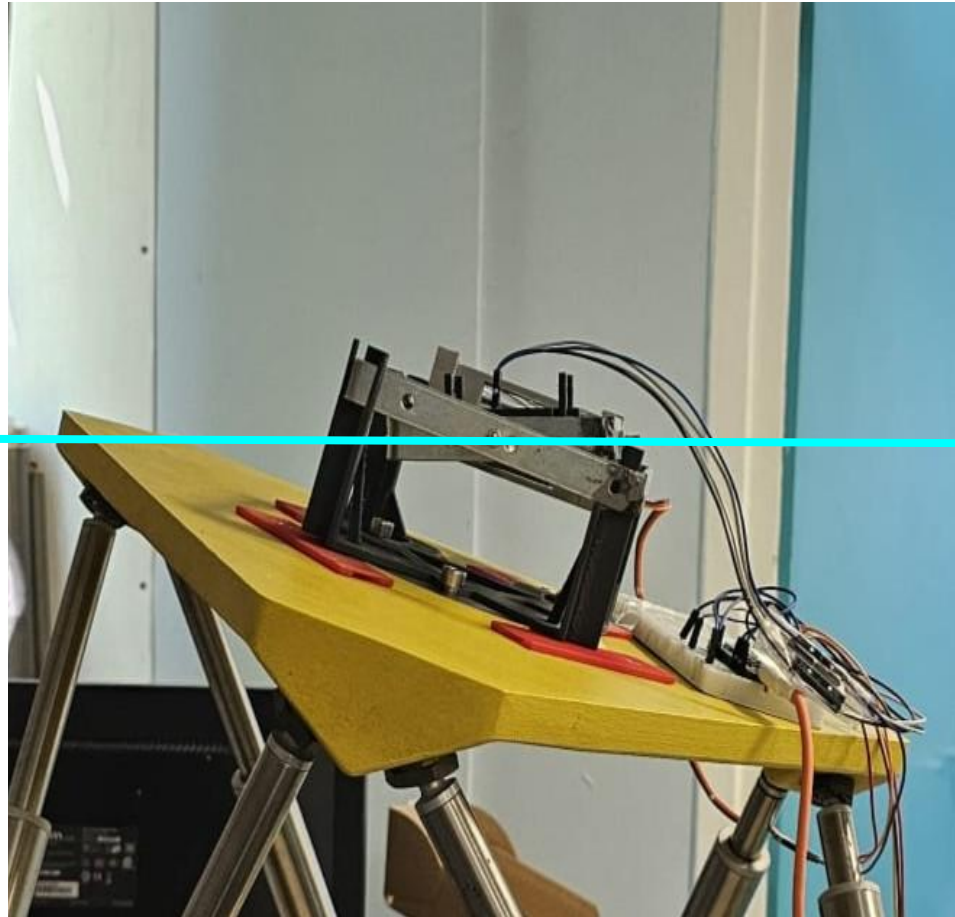
Validation théorique

- ⚡ Amélioration de la correction : moins de bruit
- ✓ Analyse possible : repérage de maximum et minimum

↘ Kp et Ki

↘ Fréquence de mouvement `sleep_ms(100)`

Asservissement : Boucle PID



Plateforme stabilisée

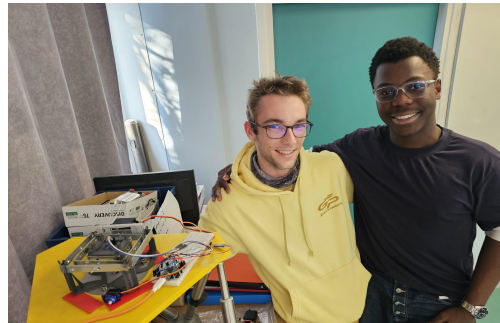
Conclusion

La plateforme multi-axes démontre l'efficacité des **boucles de rétroaction** pour la stabilisation dynamique d'un système instable.



Problèmes résolus

Démarches scientifiques, réflexions profondes, résultats probants



Perspectives

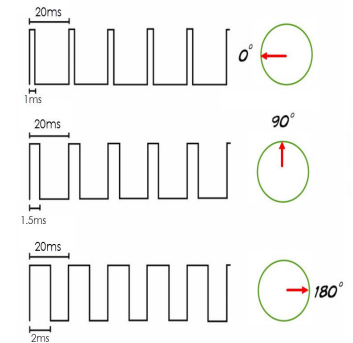
Intégration d'un 2ème axe (Lacet)

Annexe 1 : Programme principal

```
1 from machine import Pin, SoftI2C
2 from math import *
3 from time import sleep_ms, ticks_ms, ticks_diff
4 from servo3 import Servo
5 from pid2 import PID
6 from mp_simple import MPU6050
7
8 i2c = SoftI2C(scl=Pin(21), sda=Pin(20), freq=400000)
9 mpu = MPU6050(i2c)
10 servo = Servo(13)
11
12 Kp, Ki, Kd= 2.0, 0.50, 1.0
13 t0= ticks_ms()
14 alpha = 0.9
15
16 pid = PID(kp=Kp, ki=Ki, kd=Kd)
17 pid.SetOutputLimits(-90, 90)
18 pid.Setpoint = 0.0
19
20 while True:
21     pitch_gyro, pitch_acc = mpu.angle()
22     pitch_filtre = (1-alpha)*pitch_gyro + alpha*pitch_acc
23
24     pid.Input = pitch_filtre
25     correction = pid.Output
26     t = ticks_diff(ticks_ms(), t0)
27     servo.move_rad(radians(correction))
28
29     print("{} {} {} {} {} {} {} \n".format(t, pitch_filtre, radians(correction),
30                                             pitch-servo.get_current_angle(), Kp, Ki, Kd))
31     sleep_ms(100)
```

Annexe 2 : Classe de commande du servomoteur

```
1 from machine import Pin, PWM
2 import math
3 import time
4
5 class Servo:
6
7     def __init__(self, pin, min_us=500, max_us=2500, angle_range_deg=180, offset_rad=math.pi/2):
8         self.pwm = PWM(Pin(pin))
9         self.pwm.freq(50)
10
11         self.offset_rad = math.pi/2
12         self.min_us = min_us
13         self.max_us = max_us
14         self.angle_range = math.radians(angle_range_deg)
15         self.current_angle = 0
16
17     def _us_to_duty(self, us):
18         period_us = 20000
19         return int(us * 65535 / period_us)
20
21
22     def move_rad(self, angle):
23         angle = min(self.angle_range + self.offset_rad, angle) + self.offset_rad
24
25         us = self.min_us + (self.max_us - self.min_us) * (angle / self.angle_range)
26
27         self.pwm.duty_u16(self._us_to_duty(us))
28
29         self.current_angle = angle
30
31     def move_relative_rad(self, delta_rad):
32         self.move_rad(self.current_angle + delta_rad)
33
34     def get_current_angle(self):
35         return self.current_angle
```



<https://www.engineersgarage.com>

```
1 from math import *
2 from servo3 import Servo
3 from machine import Pin, PWM
4 from time import sleep
5
6 led = Pin("LED", Pin.OUT)
7 s = Servo(pin=13)
8
9 # angles en radians
10 angle_MPU = [0, math.pi/4, math.pi/2, -math.pi/4, -math.pi/2, 0]
11 for a in angle_MPU:
12     led.on()
13     s.move_rad(a)
14     print("Move to {:.2f} rad".format(s.get_current_angle()))
15     sleep(1)
16 led.off()
```

Test de fonctionnement

Annexe 3 : Gestion des données 1/2

```
1 import serial
2 import time
3 print("Initialisation système")
4 port = "COM4"
5 baud = 115200
6 print("Démarrage")
7 ser = serial.Serial(port, baud)
8 print("Go")
9
10 time.sleep(2)
11
12 C = open("C:\\XboxGames\\TIPE\\MesurePID\\CompteurPID.csv", "a")
13 C.write("allo" + "\n")
14 C.close()
15
16 C=open("C:\\XboxGames\\TIPE\\MesurePID\\CompteurPID.csv", "r")
17 L=C.readlines()
18 print(len(L))
19
20 f = open("C:\\XboxGames\\TIPE\\MesurePID\\MesurePID" + str(len(L)) + ".csv", "w")
21 print("Enregistrement... Ctrl+C pour arrêter")
22
23 while True:
24     line = ser.readline().decode().strip()
25     print(line)
26     f.write(line + "\n")
27
28
29 f.close()
30 ser.close()
```

Ecriture des données

```
14 from math import *
15 import numpy as np
16 import matplotlib.pyplot as plt
17
18 Numeros = []
19 F = []
20 Temps = []
21 A = []
22 S = []
23 SI=[]
24 C = []
25 LKp = []
26 LKi = []
27
28 for i in range(1, 22):
29     try:
30
31         F.append(np.loadtxt(f"C:\\XboxGames\\TIPE\\MesurePID\\MesurePID{i}.csv", skiprows=1))
32         Numeros.append(i)
33
34         except FileNotFoundError:
35             pass
36
37     for i in range(len(F)):
38         Temps.append(F[i][:,0]/1000)
39         A.append(F[i][:,1]*180/pi)
40
41         S.append(F[i][:, 2]*180/pi)
42         C.append(F[i][:, 3])
43         try:
44             LKp.append(F[i][:,4])
45         except IndexError:
46             LKp.append(np.array([0]))
47
48         try:
49             LKi.append(F[i][:,5])
50         except IndexError:
51             LKi.append(np.array([0]))
```

Récupération des
données sur Python

Annexe 3 : Gestion des données 2/2

```
53
54     for i in range(len(Numeros)):
55         plt.figure(i)
56         plt.title("Mesure :" + str(i+1))
57         plt.plot(Temps[i], A[i], 'b', label="Inclinaison brute", linewidth=0.5)
58         plt.plot(Temps[i], S[i], 'r', linewidth=0.5
59                 , label="Angle Servo avec Kp={}, Ki={}".format(LKp[i][0], LKi[i][0]))
60         plt.xlabel("Temps (s)")
61         plt.ylabel("Angle (deg)")
62         plt.legend(loc='upper left')
63         plt.grid()
64
65     plt.show()
66
```

Traçage des graphes